

DISEÑO DE SISTEMAS DE INFORMACIÓN

“METAMAPA”

JUSTIFICACIONES DE DISEÑO

Trabajo Práctico Anual Integrador
-2025-

Grupo N°5



ENTREGA 3: Arquitectura y Modelado en Objetos - Parte III

Entregables

1. Justificaciones de Diseño Iniciales.

FUENTE ESTÁTICA

FuenteEstaticaController expone endpoints HTTP para acceder a información estática de hechos a partir de datasets previamente cargados.

Endpoint base: /api/estatica

1. Obtener lista de hechos de un dataset

GET /api/estática/{tipo}/{path}

Descripción:

Devuelve una lista de hechos (HechoDTOOutput) contenidos en un dataset determinado por su tipo.

Parámetros de ruta:

- tipo: Tipo de dataset, representado por el enum DatasetType (por ejemplo: CSV, DB, API).
- path: Ruta relativa (o nombre) del archivo fuente dentro del tipo de dataset (ej: "desastres.csv").

PATRON FACTORY METHOD:

Recibe un parámetro (el tipo de dataset). Crea una subclase concreta (DatasetCsv, DatasetDb, etc.) Oculta la lógica de construcción al cliente. Selecciona dinámicamente la implementación de una interfaz (Dataset).

Ejemplo de llamada:

GET /api/estatica/CSV/catamarca.csv

Json de Respuesta:

```
[
  {
    "titulo": "Aluvión en Catamarca",
    "descripcion": "Un aluvión afectó el norte de Catamarca...",
    "fecha": "2023-01-15",
    "latitud": -28.4696,
    "longitud": -65.7852
  },
  ...]
```

2. Obtener un hecho específico por título

GET /api/estatica/{tipo}/{path}/{titulo}

Descripción:

Devuelve un único hecho cuyo título coincida exactamente con el valor proporcionado.

Parámetros de ruta:

- **Tipo:** Tipo de dataset (mismo que en el endpoint anterior).
- **Path:** Ruta o nombre del dataset.
- **Título:** Título exacto del hecho a buscar.

Ejemplo de llamada:

GET /api/estatica/CSV/incendios.csv/Incendio en Bariloche

Json de Respuesta:

```
{
  "titulo": "Incendio en Bariloche",
  "descripcion": "Un gran incendio forestal se desarrolló en la zona sur...",
  "fecha": "2022-11-03",
  "latitud": -41.1335,
  "longitud": -71.3103
}
```

Requisitos internos

- El servicio IHechoService es responsable de acceder a los datos y realizar el parseo de los datasets.
- El parámetro path debe coincidir con un archivo disponible en el sistema.
- Verificar que el tipo utilizado esté correctamente definido en DatasetType.
- Los títulos deben coincidir exactamente (sin tildes ni diferencias en mayúsculas/minúsculas).

FUENTE PROXY

Este controlador actúa como **proxy dinámico** hacia APIs externas de hechos, como "metamapa" o "disilab". Permite abstraer las diferencias entre APIs y unificar su acceso mediante diferentes patrones.

Endpoint base: /api/proxy

1. Obtener todos los hechos de una API externa

GET /api/proxy/{api}

Descripción:

Recupera todos los hechos disponibles en una fuente externa especificada por su nombre.

Parámetros de ruta:

- **Api:** Nombre de la fuente externa. Debe coincidir con uno de los nombres soportados por el sistema (por ejemplo: "metamapa", "disilab", etc.).

Ejemplo de llamada:

GET /api/proxy/disilab

Respuesta esperada:

```
[
  {
    "id": 1,
    "titulo": "Inundación en Córdoba",
    "descripcion": "Lluvias intensas provocaron inundaciones...",
    "fecha": "2023-04-10",
    "latitud": -31.4167,
    "longitud": -64.1833
  },
  ...
]
```

2. Obtener un hecho específico por ID desde una API externa

GET /api/proxy/{api}/{id}

Descripción:

Devuelve un hecho único consultado por su id, dentro de la fuente especificada.

Parámetros de ruta:

- **api:** Nombre de la API externa.
- **id:** Identificador numérico del hecho en la fuente externa.

Ejemplo de llamada:

GET /api/proxy/disilab/17

Respuesta esperada:

```
{
  "id": 17,
  "titulo": "Granizo en Mendoza",
```

```
"descripcion": "Un fuerte granizo afectó zonas vitivinícolas...",  
"fecha": "2022-12-05",  
"latitud": -32.8908,  
"longitud": -68.8272  
}
```

Patrones aplicados:

Adapter:

Cada API externa tiene su propia estructura (recibo DTO'S externos), necesito adaptar su respuesta a mi modelo interno (HechoDTOOutput).

Strategy:

Seleccionar dinámicamente cuál fuente usar (Disilab, MetaMapa...), por ejemplo, según un parámetro de entrada, o configuración.

Requisitos internos

- El servicio IProxyService encapsula la lógica para decidir qué fuente externa consultar en función del nombre (api).
- Las fuentes deben estar previamente configuradas y expuestas a través de HTTP por nombre.
- Las respuestas externas son adaptadas a objetos HechoDTOOutput para mantener uniformidad con otras partes del sistema.

FUENTE DINÁMICA

Esta API permite **crear, consultar, editar y eliminar** hechos reportados dinámicamente, con o sin usuario asociado (mantiene el anonimato). También permite administrar usuarios que pueden publicar y editar hechos.

Endpoints base:

- /api/dinamica
- /api/dinamica/usuarios

Endpoints - Gestión de Hechos

1. Crear hecho (sin usuario)

POST /api/dinamica

Body (JSON):

```
{  
  "titulo": "Corte eléctrico en Salta",  
  "descripcion": "Se registró un apagón en la zona",  
}
```

```
"fecha": "2025-06-01",
"lugarAcontecimiento": {
  "latitud": -24.7821,
  "longitud": -65.4232
}
}
```

Respuesta:

Hecho creado, cabe aclarar que se crea con **editable = false**.

2. Crear hecho (con usuario registrado)

POST /api/dinamica/{idUserio}

Body: Igual que en el caso anterior

```
{
  "titulo": "Corte eléctrico en Salta",
  "descripcion": "Se registró un apagón en la zona",
  "fecha": "2025-06-01",
  "lugarAcontecimiento": {
    "latitud": -24.7821,
    "longitud": -65.4232
  }
}
```

Respuesta:

Hecho creado, **editable = true** si el usuario existe.

3. Obtener todos los hechos

GET /api/dinamica

Respuesta:

Te devuelve aquellos hechos que provee la fuente dinámica

4. Obtener hecho por ID

GET /api/dinamica/{id}

Respuesta:

```
{
  "id": 1,
  "titulo": "...",
  "descripcion": "...",
  "fecha": "...",

```

```
"latitud": ...,  
"longitud": ...  
}
```

5. Editar hecho por ID

PUT /api/dinamica/{id}

Condición: Solo se puede editar si el hecho es **editable = true**.

Body (JSON): Igual a creación

```
{  
  "titulo": "Corte eléctrico en Salta",  
  "descripcion": "Se registró un apagón en la zona",  
  "fecha": "2025-06-01",  
  "lugarAcontecimiento": {  
    "latitud": -24.7821,  
    "longitud": -65.4232  
  }  
}
```

Errores posibles:

- 403 FORBIDDEN: si el hecho no es editable.
- 404 NOT FOUND: si el hecho no existe.

6. Eliminar hecho por ID

URL:

DELETE /api/dinamica/{id}

Respuesta:

Devuelve el hecho eliminado en formato DTO.

Endpoints - Gestión de Usuarios

1. Crear usuario

POST /api/dinamica/usuarios

Body (JSON):

```
{  
  "nombre": "hamze",  
  "password": "superSecreta123",  
  "edad": 25,  
}
```

```
"rol": {  
  "nombre": "Visualizador"  
}  
}
```

Respuesta:

```
{  
  "id": 7,  
  "nombre": "Juan Pérez",  
}
```

2. Obtener todos los usuarios

GET /api/dinamica/usuarios

Respuesta:

Lista de usuarios en formato DTOOutput.

3. Obtener usuario por ID

GET /api/dinamica/usuarios/{id}

Respuesta:

Te devuelve el usuario que tenga referido dicho ID

4. Editar usuario

PUT /api/dinamica/usuarios/{id}

Body: Igual que creación.

```
{  
  "nombre": "hamze",  
  "password": "superSecreta123",  
  "edad": 25,  
  "rol": {  
    "nombre": "Visualizador"  
  }  
}
```

5. Eliminar usuario

DELETE /api/dinamica/usuarios/{id}

Que te retorna el usuario eliminado

SERVICIO AGREGADOR

El módulo de Servicio Agregador permite la gestión de hechos a través de diferentes fuentes, integrando datos de diversas APIs y fuentes de datos estáticas. Además permite proveer servicio a las diferentes solicitudes de eliminación cargadas en el sistema

Endpoint Base: /api/colecciones
/api/solicitudes

El controlador **ColeccionController** expone endpoints HTTP para gestionar colecciones de hechos, tanto en su estructura básica como en sus fuentes asociadas y hechos contenidos. También se prevé extender su funcionalidad a navegación filtrada y curada sobre las colecciones.

Endpoint base: /api/colecciones

1. Operaciones CRUD sobre Colecciones

Crear colección

POST /api/colecciones

Recibe una colección en formato ColeccionDTOInput, la convierte a entidad de dominio y la guarda.

Respuesta: DTOOutput con la colección creada.

Obtener todas las colecciones

GET /api/colecciones

Devuelve la lista completa de colecciones almacenadas, en formato ColeccionDTOOutput.

Obtener colección por ID

GET /api/colecciones/{id}

Retorna una única colección, si existe, identificada por su ID.

Actualizar colección

PUT /api/colecciones/{id}

Actualiza una colección existente con los nuevos datos provistos en el cuerpo del request.

Eliminar colección

DELETE /api/colecciones/{id}

Elimina la colección correspondiente al ID recibido y devuelve su DTO de salida.

2. Hechos de una colección

Obtener hechos de una colección

GET /api/colecciones/{id}/hechos

Devuelve todos los hechos asociados a una colección en formato HechoDTOOutput.

Internamente, los hechos pueden provenir de distintas fuentes estáticas, dinámicas o proxies. Esta lógica está encapsulada dentro del servicio IColeccionService.

3. Modificar algoritmo de consenso

Actualizar algoritmo

PUT /api/colecciones/{id}/algoritmo-consenso?algoritmo={valor}

Permite modificar el tipo de algoritmo de consenso que se utiliza para determinar qué hechos son válidos dentro de la colección.

Uso de Enum: TiposAlgoritmos define los algoritmos disponibles (por ejemplo, mayoría simple, unanimidad, puntuación ponderada, etc.).

4. Gestión de fuentes dentro de una colección

Agregar fuente

POST /api/colecciones/{id}/fuentes/{fuenteld}

Agrega una nueva fuente de hechos a una colección. La fuente se describe en

FuenteDTOInput, que incluye:

- Tipo de fuente (estática, dinámica, proxy)
- Ruta o identificador
- Información adicional de la fuente

Quitar fuente

DELETE /api/colecciones/{id}/fuentes/{fuenteld}

Elimina una fuente específica asociada a una colección.

Patrones aplicados:

- Strategy: Se emplea para determinar la lógica de navegación o consenso por tipo de colección.
- Factory/Polimorfismo: Las fuentes se tratan como objetos polimórficos (FuenteAlt) y su comportamiento varía según su tipo (TipoFuente).

5. Navegación curada y filtrada (en desarrollo - todo)

Se prevé extender la funcionalidad del controlador con:

- Navegación curada: Devuelve hechos preseleccionados por la colección (por lógica definida internamente).
- Navegación filtrada: Permite al usuario filtrar hechos según parámetros como categoría, fecha de inicio y fin, etc.

SOLICITUDES

El controlador SolicitudController expone endpoints HTTP para la gestión de solicitudes de eliminación de hechos. Estas solicitudes pueden ser aprobadas o denegadas por un administrador o moderador del sistema.

Endpoint base: /api/solicitudes

1. Solicitudes de eliminación de hechos

Aprobar solicitud

PUT /api/solicitudes/eliminación/{solicitudId}/aprobar

Marca la solicitud como aprobada e inicia el proceso de eliminación del hecho correspondiente.

Denegar solicitud

PUT /api/solicitudes/eliminacion/{solicitudId}/denegar

Marca la solicitud como denegada. El hecho permanece sin modificaciones.

Obtener todas las solicitudes

GET /api/solicitudes/eliminacion

Devuelve la lista de todas las solicitudes de eliminación, adaptadas como SolicitudEliminacionDTOOutput.

Crear solicitud de eliminación

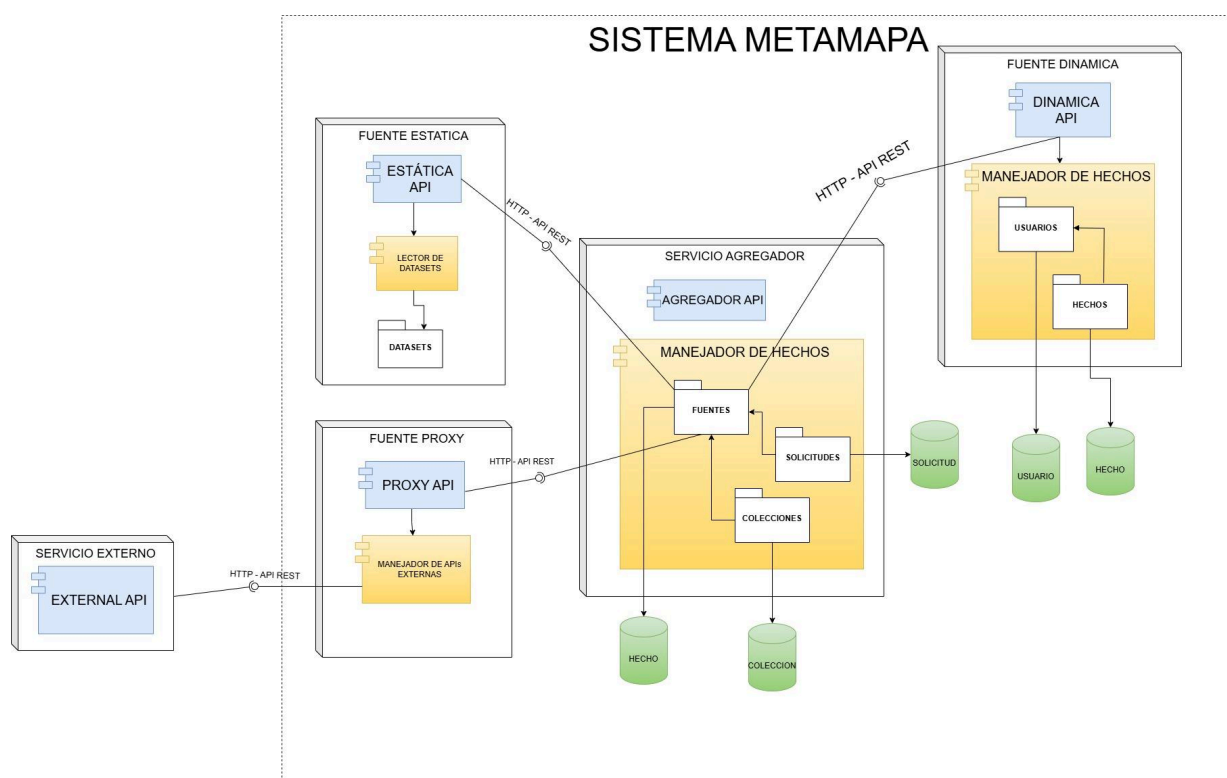
POST /api/solicitudes/eliminacion

Crea una nueva solicitud, transformando el DTO de entrada en una entidad de dominio (SolicitudEliminacion).

Requisitos internos y diseño

- El servicio ISolicitudEliminacionService encapsula toda la lógica de validación y decisión respecto a las solicitudes.
- Se utiliza un mapper (SolicitudMapper) para transformar entre entidades de dominio y DTOs.
- El diseño separa responsabilidades entre el controlador (exposición de endpoints), el servicio (lógica de negocio), y los mappers (transformación de objetos).
- El hecho asociado a una solicitud de eliminación se elimina sólo si la solicitud es aprobada.

2. Diagrama de Arquitectura.



3. Diseño UX UI

Para el diseño decidimos que sea minimalista, con elementos fáciles de encontrar. Creemos que el foco de esta app/página web son los hechos, entonces decidimos profundizar en ellos. El flujo de frames es el siguiente:

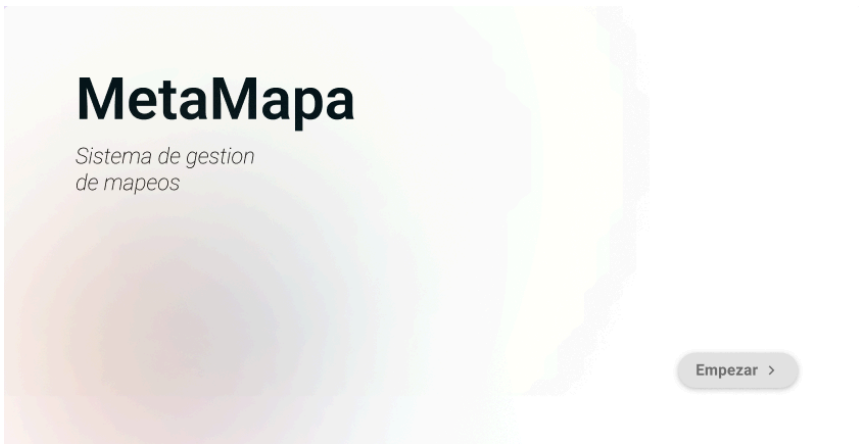
Empieza en el inicio donde el usuario aprieta un botón para comenzar. Luego de eso se despliega un Login que el usuario puede decidir no hacer (si se quiere mantener como incógnito). Después va al frame principal donde se ven los diferentes hechos (principalmente la foto característica de cada hecho). Luego si quiere el usuario puede apretar su icono para crear un hecho (si tiene permisos) o visualizar su perfil.

Para obtener más información de algún hecho solo hace falta presionarlo. Se abrirá un pop up con el nombre, la imagen, descripción etc.

Por último, en la pantalla principal también se podrán filtrar y buscar hechos con los botones respectivos.



Mostraremos algunos frames con más detalle:



4. Implementación de los requerimientos.

Link al repositorio: <https://github.com/dds-utn/2025-tpa-ma-ma-grupo-05/tree/entrega2>